

Programação Visual e Autoria Web

Estilo Arquitetural REST e
Desenvolvimento de um CRUD
com o Framework Angular

Universidade Federal do Rio Grande do Norte

Recursos

- A RFC 7231 3 especifica um conjunto de 8 métodos os quais podemos utilizar para a criação de uma API RESTful.
- Dos 8 métodos os 5 mais usados em APIs RESTful são:
 - GET é utilizado quando existe a necessidade de se obter um recurso.
 - POST é utilizado para processamento de um recurso a partir do uso de uma representação.
 - PUT é utilizado como forma de atualizar ou inserir um determinado recurso.
 - PATCH executa uma atualização parcial de um recurso. O corpo da solicitação especifica o conjunto de alterações a serem aplicadas ao recurso.
 - DELETE tem como finalidade a remoção de um determinado recurso

GET

- Um método GET bem-sucedido geralmente retorna o código de status HTTP 200 (OK).
- Se o recurso não puder ser encontrado, o método deve retornar 404 (Not Found).
- Se a solicitação foi atendida, mas não há corpo de resposta incluído na resposta HTTP, ela deve retornar o código de status HTTP 204 (sem conteúdo);
 - Por exemplo, uma operação de pesquisa que não produz correspondências pode ser implementada com esse comportamento.

POST

- Se um método POST criar um novo recurso, ele retornará o código de status HTTP 201 (Criado). O URI do novo recurso é incluído no cabeçalho Location da resposta.
 - O corpo da resposta contém uma representação do recurso.
- Se o método fizer algum processamento, mas não criar um novo recurso, o método poderá retornar o código de status HTTP 200 e incluir o **resultado da operação no corpo da resposta**.
- Como alternativa, se não houver resultado a ser retornado, o método pode retornar o código de status HTTP 204 (sem conteúdo) sem corpo de resposta.
- Se o cliente inserir dados inválidos na solicitação, o servidor deverá retornar o código de status HTTP 400 (Solicitação inválida). O corpo da resposta pode conter informações adicionais sobre o erro ou um link para um URI que fornece mais detalhes.

PUT

- Se um método PUT criar um novo recurso, ele retornará o código de status HTTP 201 (Criado), como no método POST.
- Se o método atualizar um recurso existente, ele retornará 200 (OK) ou 204 (Sem conteúdo).
- Em alguns casos, pode não ser possível atualizar um recurso existente. Nesse caso, considere retornar o código de status HTTP 409 (Conflito).
- Considere utilizar HTTP PUT para implementar atualizações em lote para vários recursos em uma coleção. A solicitação PUT deve especificar o URI da coleção e o corpo da solicitação deve especificar os detalhes dos recursos a serem modificados. Essa abordagem pode melhorar o desempenho.

DELETE

- Se a operação de exclusão for bem-sucedida, o servidor da Web deve responder com o código de status HTTP 204 (sem conteúdo), indicando que o processo foi tratado com sucesso, mas que o corpo da resposta não contém mais informações.
- Se o recurso não existir, o servidor web pode retornar HTTP 404 (não encontrado).

Filtrar e paginar dados

- A exposição de uma coleção de recursos por meio de um único URI pode fazer com que os aplicativos busquem grandes quantidades de dados quando apenas um subconjunto das informações é necessário.
 - Por exemplo, suponha que um aplicativo cliente precise encontrar todos os pedidos com um custo acima de um valor específico. Ele pode recuperar todos os pedidos do URI /pedidos e, em seguida, filtrar esses pedidos no lado do cliente. Claramente, este processo é altamente ineficiente.
- A API pode permitir a passagem de um filtro na string de consulta do URI, como /pedidos?minCusto=n. A API da Web é responsável por analisar e manipular o parâmetro minCusto na string de consulta e retornar os resultados filtrados no lado do servidor.

Filtrar e paginar dados

- Solicitações GET sobre recursos de coleção podem retornar um grande número de itens. Você deve projetar uma API da Web para limitar a quantidade de dados retornados por qualquer solicitação única.
- É importante oferecer suporte a strings de consulta que especificam o número máximo de itens a serem recuperados e um deslocamento inicial na coleção.
 - Impor um limite máximo no número de itens devolvidos, para ajudar a evitar ataques de negação de serviço. Para auxiliar os aplicativos clientes, as solicitações GET que retornam dados paginados também devem incluir alguma forma de metadados que indique o número total de recursos disponíveis na coleção.

Revisão - Components

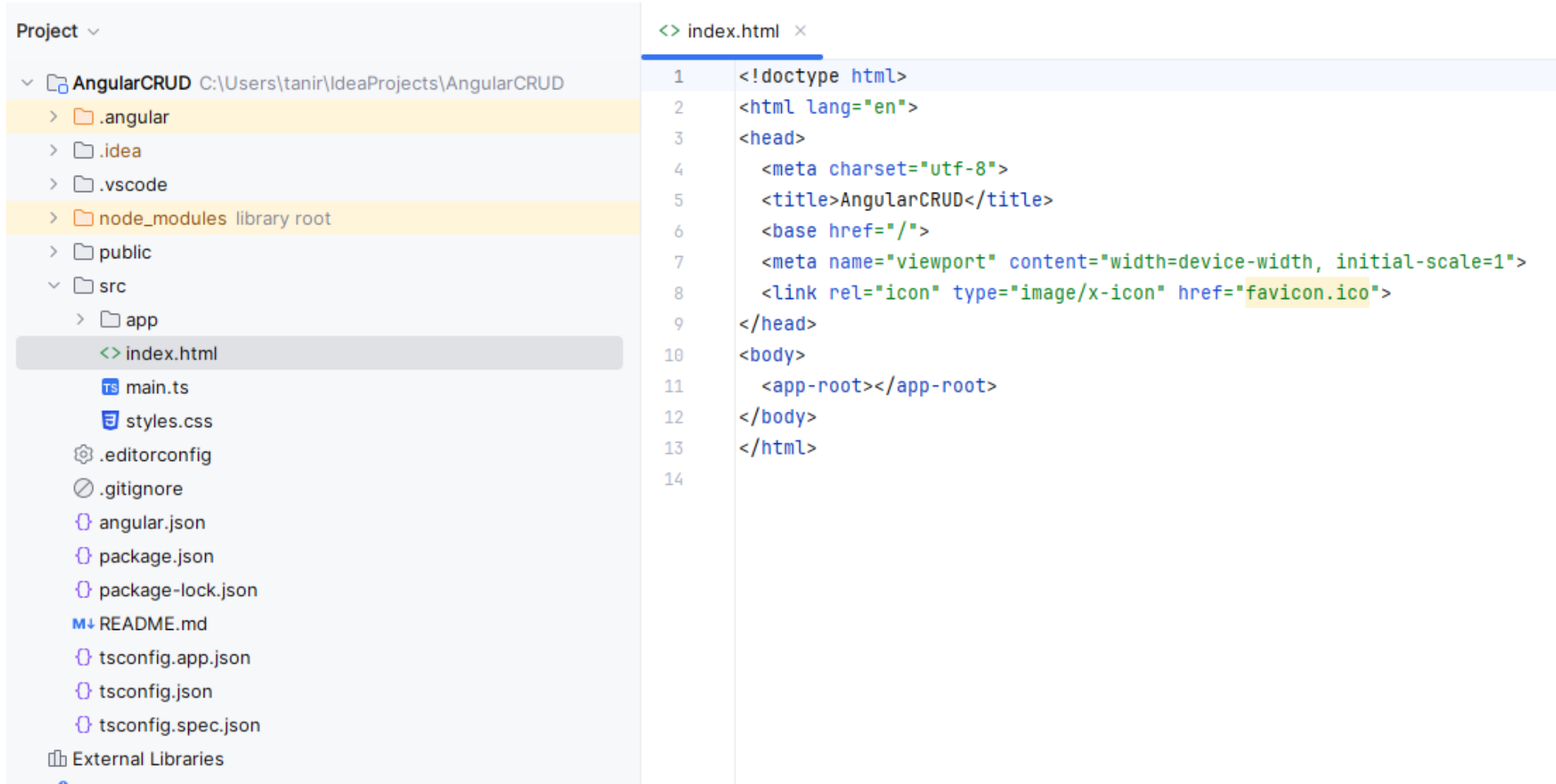
- Cada aplicação Angular possui pelo menos um **Component**.
 - Root component conecta uma hierarquia de componentes com o Document Object Model (DOM) da página.
- Cada **Component** define uma classe que contém dados e lógica da aplicação e está associada a um template HTML que define uma View a ser exibida.
- A anotação `@Component()` identifica a classe imediatamente abaixo dela como um **Component** e fornece o template e os metadados específicos do **Component** relacionado.

Services

- Service é uma categoria ampla que engloba qualquer valor, função ou recurso que uma aplicação precisa.
- Um Service é tipicamente uma classe com um propósito estreito e bem definido
- O Angular distingue componentes de serviços para aumentar a modularidade e a reutilização.
- Ao separar a funcionalidade relacionada à visualização de um componente de outros tipos de processamento, é possível tornar suas classes de Components enxutas e eficientes.
 - Um component deve ser usado para permitir a experiência do usuário
 - Um component deve apresentar propriedades e métodos para data binding, a fim de mediar entre a View (renderizada pelo template) e a lógica da aplicação.
 - Um componente pode delegar determinadas tarefas aos Services, como buscar dados do servidor
- Ao definir essas tarefas de processamento em uma classe de Service injetável, tais tarefas ficam disponíveis para qualquer Component.

Desenvolvimento de um projeto simples

```
$ npx ng new angular-produto-front
```



The image shows a screenshot of an IDE interface. On the left, the 'Project' sidebar displays the file structure of a new Angular project named 'AngularCRUD'. The files listed include configuration files like .angular, .idea, .vscode, node_modules, public, src, app, index.html, main.ts, styles.css, .editorconfig, .gitignore, angular.json, package.json, package-lock.json, README.md, tsconfig.app.json, tsconfig.json, tsconfig.spec.json, and External Libraries. The 'index.html' file is selected and highlighted. On the right, the editor shows the content of 'index.html', which is a standard HTML boilerplate for an Angular application. The code includes a doctype declaration, a lang attribute set to 'en', a head section with meta tags for charset, title, base href, and viewport, and a link to a favicon. The body section contains an 'app-root' tag.

```
<?index.html x>
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>AngularCRUD</title>
6    <base href="/">
7    <meta name="viewport" content="width=device-width, initial-scale=1">
8    <link rel="icon" type="image/x-icon" href="favicon.ico">
9  </head>
10 <body>
11   <app-root></app-root>
12 </body>
13 </html>
14
```

Adicionar Angular Material ao projeto

● <https://material.angular.io/guide/getting-started>

```
ng add @angular/material
```

```
PS D:\workspace\VsCode\Angular\angular-produto-front\angular-produto-front> ng add @angular/material
```

```
i Using package manager: npm
```

```
✓ Found compatible package version: @angular/material@13.3.7.
```

```
✓ Package information loaded.
```

```
The package @angular/material@13.3.7 will be installed and executed.
```

```
Would you like to proceed? Yes
```

```
✓ Package successfully installed.
```

```
? Choose a prebuilt theme name, or "custom" for a custom theme: Indigo/Pink
```

```
[ Preview: https://material.angular.io?theme=indigo-pink ]
```

```
? Set up global Angular Material typography styles? Yes
```

```
? Set up browser animations for Angular Material? Yes
```

```
UPDATE package.json (1150 bytes)
```

```
✓ Packages installed successfully.
```

```
UPDATE src/app/app.module.ts (502 bytes)
```

```
UPDATE angular.json (3461 bytes)
```

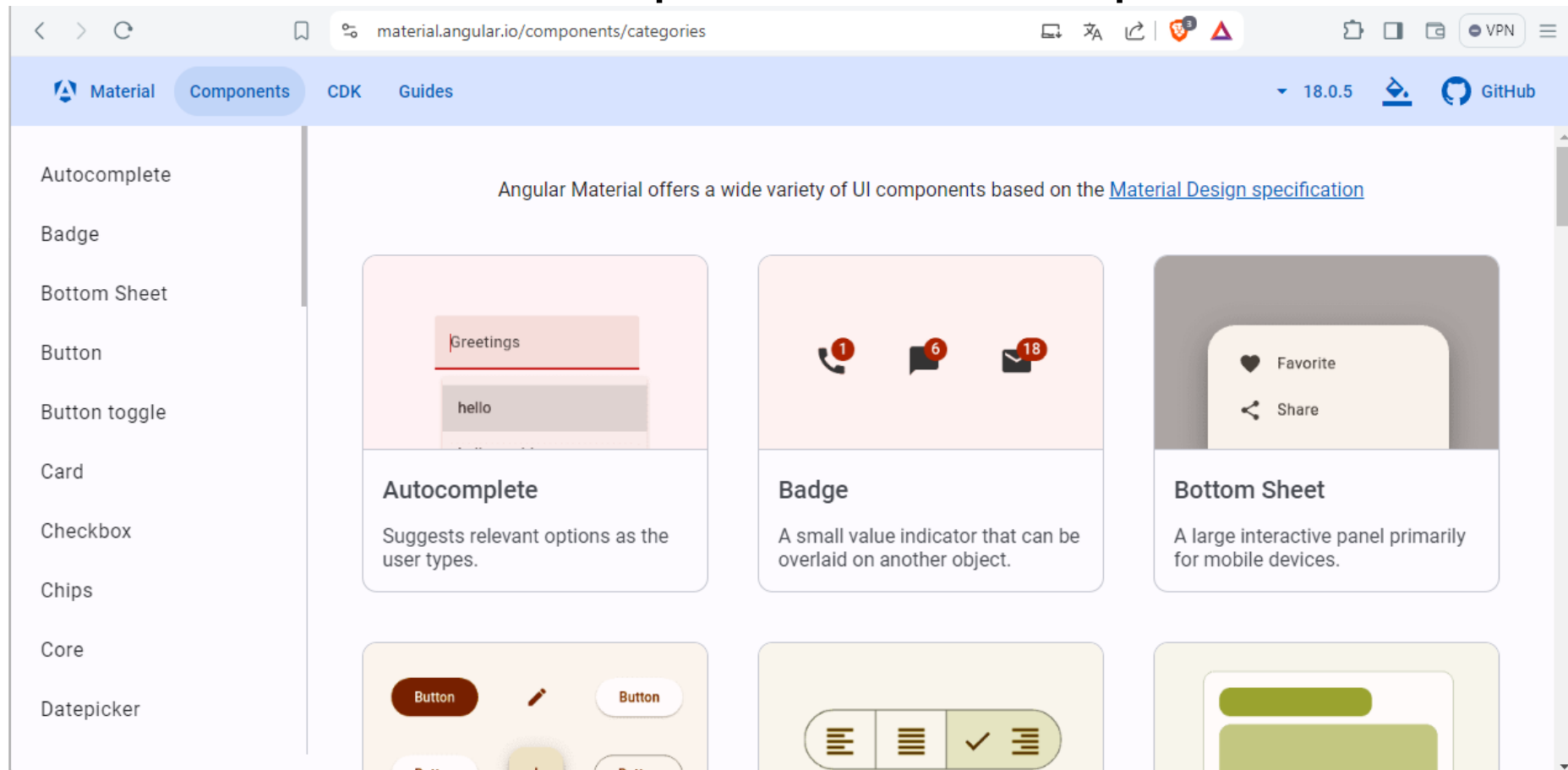
```
UPDATE src/index.html (587 bytes)
```

```
UPDATE src/styles.scss (181 bytes)
```

```
PS D:\workspace\VsCode\Angular\angular-produto-front\angular-produto-front> █
```

Componentes Angular Material

- Lembrar de realizar o import do module que será utilizado.



Componentes Angular Material

- Lembrar de realizar o import do module que será utilizado.

The screenshot shows the Angular Material API documentation for the Button component. The top navigation bar includes 'Material', 'Components' (selected), 'CDK', and 'Guides'. The version '18.0.5' and a 'GitHub' link are also visible. The left sidebar lists various components, with 'Button' highlighted. The main content area has tabs for 'OVERVIEW', 'API' (selected), and 'EXAMPLES'. The 'API' tab displays the 'API reference for Angular Material button'. It includes an import statement: `import {MatButtonModule} from '@angular/material/button';`. Below this, the 'Directives' section is shown. The 'MatButton' section describes the Material Design button component and provides a link to the Material Design specification. It also lists the selectors: `button[mat-button]`, `button[mat-raised-button]`, `button[mat-flat-button]`, and `button[mat-stroked-button]`. A right sidebar lists the component's hierarchy, including 'Directives' (MatButton, MatAnchor, MatFabButton, MatMiniFabButton, MatFabAnchor extends MatAnchor, MatMiniFabAnchor extends MatAnchor, MatIconButton, MatIconAnchor), 'Interfaces' (MatButtonConfig, MatFabDefaultOptions), and 'Constants' (MAT_BUTTON_CONFIG, MAT_FAB_DEFAULT_O...).

Componentes Angular Material

- Lembrar de realizar o import do module que será utilizado.

```
import { Component, signal } from '@angular/core';  
import { RouterOutlet } from '@angular/router';  
import { MatButtonModule } from '@angular/material/button';
```

```
@Component({  
  selector: 'app-root',  
  imports: [RouterOutlet, MatButtonModule],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css'],  
})  
export class App {  
  protected readonly title = signal('crud-produtos-app');  
}
```

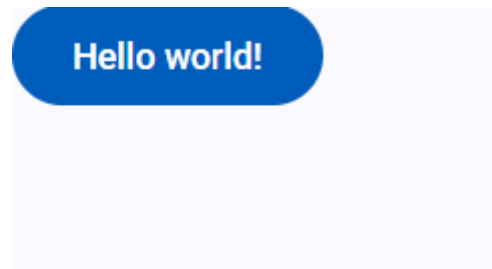
Componentes Angular Material

- Lembrar de realizar o import do module que será utilizado.

```
<button mat-flat-button>Hello world!</button>
```


Componentes Angular Material

- Lembrar de realizar o import do module que será utilizado.



Adicionar uma ToolBar para a aplicação

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, MatToolbarModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('crud-produtos-app');
}
```



Adicionar uma ToolBar para a aplicação

- Se achar necessário você pode customizar a cor da barra através do SCSS editando o arquivo custom-theme.scss na raiz do projeto.

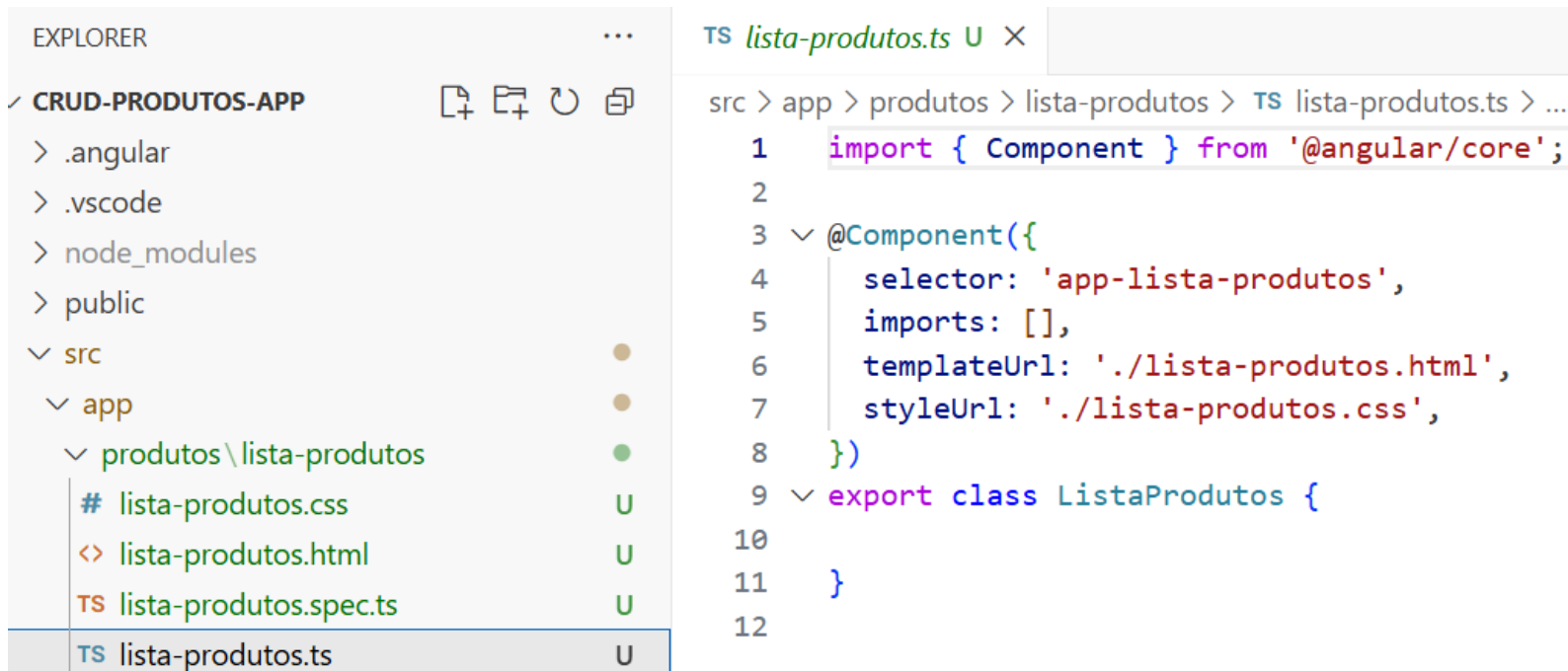
```
// Customize the entire app. Change :root to
your selector if you want to scope the styles.
:root {
  @include mat.toolbar-overrides(
    (
      container-background-color: orange,
      container-text-color: red,
    )
  );
}
```

Criação de um component ListaProdutos

- Será criado um componente de ListaProdutos que possui os templates para view, estilos, testes e comportamento.

```
ng generate component /produtos/ListaProdutos
```

```
ng g c /produtos/ListaProdutos
```



```
EXPLORER
CRUD-PRODUTOS-APP
  > .angular
  > .vscode
  > node_modules
  > public
  > src
    > app
      > produtos\lista-produtos
        # lista-produtos.css
        <> lista-produtos.html
        TS lista-produtos.spec.ts
        TS lista-produtos.ts

TS lista-produtos.ts
src > app > produtos > lista-produtos > TS lista-produtos.ts > ...
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-lista-produtos',
5    imports: [],
6    templateUrl: './lista-produtos.html',
7    styleUrls: ['./lista-produtos.css'],
8  })
9  export class ListaProdutos {
10
11  }
12
```

Routes

- Routes (ou Roteamento) é uma funcionalidade essencial em aplicações Angular, permitindo navegar entre diferentes componentes ou páginas dentro da aplicação.
- É gerenciado pelo Angular Router, um módulo poderoso que facilita:
 - Definição de caminhos (URLs).
 - Passagem de parâmetros entre componentes.
 - Controle de acesso com Guards.
- Benefícios do Roteamento:
 - Navegação sem recarregar a página inteira.
 - Experiência de aplicações SPA (Single Page Application).
 - Modularização e organização do código.

Configurar arquivo de rotas da aplicação

```
import { Routes } from '@angular/router';  
import { ListaProdutos } from  
  './produtos/lista-produtos/lista-produtos';  
  
export const routes: Routes = [  
  {  
    path: '',  
    component: ListaProdutos,  
  },  
];
```

app.component.html

```
<mat-toolbar>Listagem de Produtos</mat-toolbar>  
<router-outlet />
```

Configurar arquivo de rotas da aplicação

- Os Components são carregados de maneira *eager* por padrão.
- Ou seja, assim que a aplicação é carregada, todos os Components também são carregados, mesmo que não sejam imediatamente necessários.
- Para aplicações grandes com muitas rotas é importante realizar o *lazy loading* — que carrega Components conforme necessário.

Configurar arquivo de rotas do módulo app

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'produtos',
    loadChildren: () =>
      import('./produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
];
```


Adicionar o conteúdo da rota na aplicação principal.

```
<mat-toolbar>  
Listagem de Produtos  
</mat-toolbar>  
<router-outlet></router-outlet>
```

Listagem de Produtos

lista-produtos works!

Criação do tipo de dado para receber produtos

- Criar uma interface produto com todos os campos/atributos presentes na entidade produto.
- Sugestão: criar uma pasta model dentro do módulo produtos e criar a interface dentro dessa pasta

```
ng g interface produtos/model/produto
```

```
export interface Produto {  
  id: number,  
  nome: string,  
  descricao: string  
}
```

Adicionar o componente de tabela para listagem de produtos

● Importar módulo da tabela

The screenshot shows the Material Design Components documentation page for the `mat-table` component. The page is titled "Overview" and includes a sidebar with a list of components: Menu, Paginator, Progress bar, Progress spinner, Radio button, Ripples, Select, Sidenav, Slide toggle, Slider, and SnackBar. The main content area is divided into three tabs: OVERVIEW, API, and EXAMPLES. The OVERVIEW tab is active, showing the following text:

The `mat-table` provides a Material Design styled data-table that can be used to display rows of data.

This table builds on the foundation of the CDK data-table and uses a similar interface for its data input and template, except that its element and attribute selectors will be prefixed with `mat-` instead of `cdk-`. For more information on the interface and a detailed look at how the table is implemented, see the [guide covering the CDK data-table](#).

Getting Started

Basic use of `<table mat-table>`

No.	Name	Weight	Symbol
-----	------	--------	--------

The right sidebar contains a table of contents for the `mat-table` component, including sections like "Getting Started", "Advanced data sources", "Styling Columns", "Row Templates", and "Features".

Adicionar o componente de tabela para listagem de produtos

- Editar component ListaProduto para realizar o import

```
import { Component } from '@angular/core';
import { MatTableModule } from '@angular/material/table';

@Component({
  selector: 'app-lista-produtos',
  imports: [MatTableModule],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {}
```

Adicionar o componente de tabela para listagem de produtos

- Adicionar a tabela ao View do component Produto (listaproductos.component.html)
- Tal tabela precisa ser preenchida com dados que virão do component ListaProdutos.

```
<table mat-table [dataSource]="dataSource" class="mat-elevation-z8">
  <!-- Position Column -->
  <ng-container matColumnDef="position">
    <th mat-header-cell *matHeaderCellDef> No. </th>
    <td mat-cell *matCellDef="let element"> {{element.position}} </td>
  </ng-container>
  <!-- Name Column -->
  <ng-container matColumnDef="name">
    <th mat-header-cell *matHeaderCellDef> Name </th>
    <td mat-cell *matCellDef="let element"> {{element.name}} </td>
  </ng-container>
  <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
  <tr mat-row *matRowDef="let row; columns: displayedColumns;"></tr>
</table>
```

Criação do service para realizar o fetch na API

- Criar uma classe para guardar a lógica de obtenção dos dados.
- Service dentro do módulo produto
- Sugestão, criar o service dentro da pasta service

```
ng generate service produtos/service/produtos  
ng g s produtos/service/produtos
```



Criação do service para realizar o fetch na API

- Criar métodos do CRUD
- Implementar o método list() para realizar o fetch
- Uso da biblioteca HttpClient no service

```
import { HttpClient } from '@angular/common/http'
```

- Uso de injeção de construtor para obtenção da instancia do HttpClient
- Adicionar o HttpClient provider no app.config.ts

Criação do service para realizar o fetch na API

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideAnimationsAsync(),
    provideHttpClient()
  ]
};
```


Criação do service para realizar o fetch na API

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { first, Observable, tap } from 'rxjs';

import { Produto } from '../model/produto';

@Injectable({
  providedIn: 'root'
})
export class Produtos {

  private API_URL = "http://localhost:3000/produtos"

  constructor(private httpClient:HttpClient) { }

  list():Observable<Produto[]>{
    return this.httpClient.get<Produto[]>(this.API_URL)
      .pipe(
        first(),
        tap(p => console.log(p))
      )
  }
}
```

Atualização do component para utilizar o service

```
import { Component } from '@angular/core';
import { MatTableModule } from '@angular/material/table';
import { Observable } from 'rxjs';
import { Produto } from '../model/produto';
import { ProdutosService } from '../service/produtos.service';

@Component({
  selector: 'app-lista-produtos',
  standalone: true,
  imports: [MatTableModule],
  templateUrl: './lista-produtos.component.html',
  styleUrls: ['./lista-produtos.component.css']
})
export class ListaProdutos {
  produtos$: Observable<Produto[]>;
  produtos_array: Produto[] = [];
  displayedColumns = ['nome', 'descricao'];
  constructor(private produtosService: ProdutosService) {
    produtosService.list().subscribe(p => this.produtos_array = p)
    this.produtos$ = produtosService.list();
    console.log(produtosService.list());
  }
}
```

Atualização da table para conforme o component

```
<table mat-table [dataSource]="produtos_array" class="mat-elevation-z8" style="width: 100%">
  <!-- Nome Column -->
  <ng-container matColumnDef="nome">
    <th mat-header-cell *matHeaderCellDef>Nome</th>
    <td mat-cell *matCellDef="let p">{{ p.nome }}</td>
  </ng-container>

  <!-- Descricao Column -->
  <ng-container matColumnDef="descricao">
    <th mat-header-cell *matHeaderCellDef>Descricao</th>
    <td mat-cell *matCellDef="let p">{{ p.descricao }}</td>
  </ng-container>

  <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
  <tr mat-row *matRowDef="let row; columns: displayedColumns"></tr>
</table>
```

Pipes

- Pipes são ferramentas que transformam dados no template, permitindo que você exiba informações de maneira mais legível ou formatada para os usuários.
- Eles são uma maneira conveniente de aplicar transformações diretamente no HTML, sem a necessidade de realizar essas alterações no componente.
- Características dos Pipes
 - Transformação de dados: Pipes pegam dados de entrada e os transformam em uma saída formatada.
 - Uso em templates: São utilizados diretamente nas expressões dos templates, usando a sintaxe `{{ valor | nomeDoPipe }}`.
 - Encadeáveis: É possível usar vários pipes em sequência, como em `{{ valor | pipe1 | pipe2 }}`.
 - Customizáveis: Além dos pipes fornecidos pelo Angular, você pode criar pipes personalizados.

Exemplos de Pipes

- CurrencyPipe: Formata um número como uma moeda.

```
{{ 1234.5 | currency:'USD' }} <!-- Saída: $1,234.50 -->
```

- DatePipe: Formata datas de maneira específica.

```
{{ '2025-01-13' | date:'longDate' }} <!-- Saída: January 13, 2025 -->
```

- UpperCasePipe e LowerCasePipe: Convertem strings para maiúsculas ou minúsculas.

```
{{ 'angular' | uppercase }} <!-- Saída: ANGULAR -->
```

Exemplos de Pipes

- **JsonPipe:** Converte um objeto em uma string JSON formatada.

```
{{ { name: 'Angular' } | json }} <!-- Saída: {"name":"Angular"} -->
```

- **AsyncPipe:** Trabalha com observáveis e promessas, automaticamente se inscrevendo e exibindo o valor resolvido.

```
<div>{{ data$ | async }}</div>
```

Uso de @if e async pipe para alternar entre templates exibidas

```
import { Component } from '@angular/core';
import { MatTableModule } from '@angular/material/table';
import { Observable, delay } from 'rxjs';
import { Produto } from '../model/produto';
import { ProdutosService } from '../service/produtos.service';
import { AsyncPipe } from '@angular/common';

@Component({
  selector: 'app-lista-produtos',
  standalone: true,
  imports: [MatTableModule, AsyncPipe],
  templateUrl: './lista-produtos.component.html',
  styleUrls: ['./lista-produtos.component.css']
})
export class ListaProdutos {
  produtos$: Observable<Produto[]>;
  produtos_array: Produto[] = [];
  displayedColumns = ['nome', 'descricao'];
  constructor(private produtosService: ProdutosService) {
    produtosService.list().subscribe(p => this.produtos_array = p)
    this.produtos$ = produtosService.list().pipe(delay(3000));
    console.log(produtosService.list());
  }
}
```

```
@if (produtos$ | async; as lista_produtos_local) {
<table mat-table [dataSource]="lista_produtos_local"
  class="mat-elevation-z8" style="width: 100%">
  <!-- Nome Column -->
  <ng-container matColumnDef="nome">
    <th mat-header-cell *matHeaderCellDef>Nome</th>
    <td mat-cell *matCellDef="let p">{{ p.nome }}</td>
  </ng-container>
  <!-- Descricao Column -->
  <ng-container matColumnDef="descricao">
    <th mat-header-cell *matHeaderCellDef>Descrição</th>
    <td mat-cell *matCellDef="let p">{{ p.descricao }}</td>
  </ng-container>
  <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
  <tr mat-row *matRowDef="let row; columns: displayedColumns"></tr>
</table>
}@else {
<p>Carregando...</p>
}
```

Tratamento de erros caso a API esteja indisponível

- Adicionando um Component MatSnackBar ao ListaProdutos para exibir a mensagem de erro.

```
{...}  
import { MatSnackBarModule } from '@angular/material/snack-bar';  
  
@Component({  
  selector: 'app-lista-produtos',  
  standalone: true,  
  imports: [MatTableModule, AsyncPipe, MatSnackBarModule],  
  templateUrl: './lista-produtos.component.html',  
  styleUrls: ['./lista-produtos.component.css']  
})  
export class ListaProdutos {  
  produtos$: Observable<Produto[]>;  
  produtos_array: Produto[] = [];  
  displayedColumns = ['nome', 'descricao']  
  constructor(private produtosService: ProdutosService) {  
    produtosService.list().subscribe(p => this.produtos_array = p)  
    this.produtos$ = produtosService.list().pipe(delay(3000));  
    console.log(produtosService.list());  
  }  
}
```



```

import { Component } from '@angular/core';
import { MatTableModule } from '@angular/material/table';
import { Observable, catchError, delay, of } from 'rxjs';
import { Produto } from '../model/produto';
import { ProdutosService } from '../service/produtos.service';
import { AsyncPipe } from '@angular/common';
import { MatSnackBar, MatSnackBarModule } from '@angular/material/snack-bar';

```

```

@Component({
  selector: 'app-lista-produtos',
  standalone: true,
  imports: [MatTableModule, AsyncPipe, MatSnackBarModule],
  templateUrl: './lista-produtos.component.html',
  styleUrls: ['./lista-produtos.component.css']
})
export class ListaProdutos {
  produtos$: Observable<Produto[]>;
  produtos_array: Produto[] = [];
  displayedColumns = ['nome', 'descricao'];
  constructor(private produtosService: Produtos, public _snackBar: MatSnackBar) {
    produtosService.list().subscribe(p => this.produtos_array = p);
    this.produtos$ = produtosService.list()
      .pipe((delay(3000)))
      .pipe(
        catchError(error => {
          console.log(error);
          this.onError(error.message, '');
          return of([]);
        })
      );
  }
  onError(errorMessage: string, action: string) {
    this._snackBar.open(errorMessage, action, {
      duration: 3000
    });
  }
}

```

Adicionar botão para cadastro/editar/remover de Produtos

- Importar para o component ListaProdutos o material button e material icons e adicionar os respectivos botões

```
<div style="width: 95%">  
  <button mat-fab style="float: right">  
    <mat-icon>add</mat-icon>  
  </button>  
</div>
```

```
import { MatButtonModule } from '@angular/material/button';  
import { MatIconModule } from '@angular/material/icon';
```

```
@Component({  
  selector: 'app-lista-produtos',  
  standalone: true,  
  imports: [MatTableModule, AsyncPipe, MatSnackBarModule, MatButtonModule,  
    MatIconModule],
```

Adicionar botão para cadastro/editar/remover de Produtos

- Importar para o módulo Produtos o material button e material icons e adicionar os respectivos botões

```
<!-- Action Column -->
<ng-container matColumnDef="acao">
  <th mat-header-cell *matHeaderCellDef>Ação</th>
  <td mat-cell *matCellDef="let p">
    <button mat-icon-button color="primary" aria-label="Editar">
      <mat-icon>edit</mat-icon>
    </button>
    <button mat-icon-button color="warn" aria-label="Remover">
      <mat-icon>delete</mat-icon>
    </button>
  </td>
</ng-container>
```

Adicionar botão para cadastro/editar/remover de Produtos

- Atualizar o Component Produto para adicionar a coluna acao.

```
displayedColumns = ['nome', 'descricao', 'acao']
```

Criar o component do formulário cadastro de produtos

- Executar a linha de comando para a criação de um novo component dentro do module Produtos.

```
ng g c /produtos/form-produtos
```

- Criar uma rota para acessar a View do component de formulário de produto.

```
export const routes: Routes = [  
  {path: "produtos", loadComponent: () => import('./produtos/lista-produtos/lista-produtos').then(m  
=> m.ListaProdutos)},  
  {path: "produtos/new", component: FormProdutos}  
];
```

Implementar a navegação para o formulário

- Implementação do [click] no template

```
<button mat-fab style="float: right" (click)="onAdd()" color="primary">
```

- Implementação do método onAdd no component

```
import { ActivatedRoute, Router } from '@angular/router';

export class ListaProdutos {

  constructor(
    private produtosService: ProdutosService,
    public _snackBar: MatSnackBar,
    private router: Router,
    private activatedRoute: ActivatedRoute
  ) {

    onAdd(){
      this.router.navigate(['new'], {relativeTo: this.activatedRoute})
    }
  }
}
```

Implementação do component de formulário

- Adicionar as bibliotecas ReactiveFormsModule, MatFormFieldModule e MatInputModule ao component formprodutos.

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { ProdutosService } from '../service/produtos.service';
import { MatSnackBar } from '@angular/material/snack-bar';
import { Location } from '@angular/common';
import { MatButtonModule } from '@angular/material/button';
@Component({
  selector: 'app-form-produtos',
  imports: [ReactiveFormsModule, MatFormFieldModule, MatInputModule, MatButtonModule],
  templateUrl: './form-produtos.component.html',
  styleUrls: ['./form-produtos.component.css'],
})
export class FormProdutos {
```

Implementação do component de formulário

- Implementar o component de formulário

```
<form [formGroup]="form">

  <mat-form-field>
    <input matInput placeholder="Nome" formControlName="nome" />
  </mat-form-field>

  <br />
  <mat-form-field>
    <input matInput placeholder="Descrição" formControlName="descricao" />
  </mat-form-field>

  <br />

  <button mat-flat-button (click)="onSubmit()">
    Cadastrar
  </button>
  <button mat-flat-button (click)="onCancel()">Cancelar</button>
</form>
```


Implementação do component de formulário

● Implementar o component de formulário

```
export class FormProdutos {  
  form: FormGroup;  
  
  constructor(  
    private FormBuilder: FormBuilder,  
    private service: Produtos,  
    private snackBar: MatSnackBar,  
    private location: Location) {  
    this.form = this.formBuilder.group({  
      nome: [null],  
      descricao: [null]  
    })  
  }  
}
```

```
onSubmit(){  
  this.service.create(this.form.value)  
    .subscribe({  
      next: (v) => this.onSucess(),  
      error: (e) => this.snackBar.open(e, "", {duration:1000 }),  
      complete: () => console.info('complete')  
    })  
}  
  
onSucess(){  
  this.snackBar.open("Salvo!", "", {duration:1000 })  
  this.location.back();  
}  
  
onCancel(){  
  this.location.back();  
}
```

Implementar a ação de adicionar

- Atualizar o código do Service para adicionar a operação de adicionar.

```
create(p:Produto){  
    return this.httpClient.post<Produto>(this.API_URL, p);  
}
```

Deletar e Editar...

● Serviço

```
delete(id:number){  
    return this.httpClient.delete(this.API_URL+"/"+id.toString());  
}  
update(id:number, p:Produto): Observable<Produto> {  
    return this.httpClient.put<Produto>(this.API_URL + '/' + id,  
JSON.stringify(p))  
}  
getOne(id:number):Observable<Produto>{  
    return this.httpClient.get<Produto>(this.API_URL+"/"+id.toString());  
}
```

Deletar e Editar...

● Component

```
onDelete(id:number){
  this.produtosService.delete(id)
  .subscribe({
    next: (v) => console.log(v),
    error: (e) => console.log(e),
    complete: () => this.produtos$ = this.produtosService.list()
  });
}

onEdit(id:number){
  this.router.navigate([id.toString()+'/edit'], {relativeTo: this.activatedRoute})
}
```

Deletar e Editar

● Template

```
<ng-container matColumnDef="acao">
  <th mat-header-cell *matHeaderCellDef>Ação</th>
  <td mat-cell *matCellDef="let p">
    <button
      mat-icon-button
      color="primary"
      aria-label="Editar"
      (click)="onEdit(p.id)"
    >
      <mat-icon>edit</mat-icon>
    </button>
    <button
      mat-icon-button
      color="warn"
      aria-label="Remover"
      (click)="onDelete(p.id)"
    >
      <mat-icon>delete</mat-icon>
    </button>
  </td>
</ng-container>
```

Rota para componente de Edição

```
{  
  path: "produtos/edit",  
  component: FormEditProduto  
}
```

Navegação do componente de Listagem para o componente de Edição

```
<!-- Action Column -->
<ng-container matColumnDef="acao">
  <th mat-header-cell *matHeaderCellDef>Ação</th>
  <td mat-cell *matCellDef="let p">
    <button mat-icon-button color="primary" aria-label="Editar" (click)="onEdit(p.id)">
      <mat-icon>edit</mat-icon>
    </button>
    <button mat-icon-button color="warn" aria-label="Remover" (click)="onDelete(p.id)">
      <mat-icon>delete</mat-icon>
    </button>
  </td>
</ng-container>
```

```
onEdit(produto_id: number) {
  this.router.navigate(['edit'], { relativeTo: this.activatedRoute, queryParams: { id: produto_id } })
}
```

Formulário do Edit

```
<form [formGroup]="form">
  <mat-form-field>
    <input matInput placeholder="Nome" formControlName="nome" />
  </mat-form-field>
  <br />
  <mat-form-field>
    <input matInput placeholder="Descrição" formControlName="descricao" />
  </mat-form-field>
  <br />
  <button mat-flat-button>
    Cadastrar
  </button>
  <button mat-flat-button>Cancelar</button>
</form>
```



```
import { Component } from '@angular/core';
import { Produto } from '../model/produto';
import { ProdutoService } from '../service/produto.service';
import { ActivatedRoute } from '@angular/router';
import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatSnackBar } from '@angular/material/snack-bar';
import { Location } from '@angular/common';
import { Observable } from 'rxjs';

@Component({
  selector: 'app-form-edit-produto',
  imports: [ReactiveFormsModule, MatFormFieldModule, MatInputModule, MatButtonModule],
  templateUrl: './form-edit-produto.component.html',
  styleUrls: ['./form-edit-produto.component.css']
})
export class FormEditProduto {
}
```

```

export class FormEditProduto {
  p$: Observable<Produto>;
  form: FormGroup;
  constructor(private servicoProduto: ProdutoService, private route: ActivatedRoute,
private FormBuilder: FormBuilder, private location: Location) {
    let id = this.route.snapshot.queryParamMap.get('id');
    this.p$ = this.servicoProduto.getOne(id!);
    // Inicialize o formulário
    this.form = this.formBuilder.group({
      nome: [null],
      descricao: [null]
    });
    // Carregar os dados da API no formulário
    this.p$.subscribe(j => {
      this.form.patchValue({
        nome: j.nome,
        descricao: j.descricao
      });
    });
  }
}

```

Referências

- <https://material.angular.io/>
- <https://angular.io/>